

Figo.

Build & Use Guide — from an empty desk to a little face that watches the market.

Firmware 1.0.0 · Waveshare ESP32-C6-Touch-LCD-1.47

Open source · MIT licence

Contents

01 What Figo is

02 What you need

03 Printing the stand

04 Arduino IDE and the ESP32 board package

05 Library versions that work

06 The two settings everyone gets wrong

07 File placement

08 Flashing

09 Assembling the board into the stand

10 First switch-on and joining WiFi

11 Setting up screens from your phone

12 Settings, faces and the star

13 What each face means

14 Troubleshooting

15 Putting Figo back to factory settings

16 Appendix: the web API and the settings format

CHAPTER 1

What Figo is

Figo is a small screen that sits on your desk and watches the market for you.

It shows live prices on a carousel of screens: a stock index, a crypto pair, gold, a Fear & Greed gauge — whatever you pick. Between the prices it shows a face. The face reacts to how one instrument you choose has moved over the last 24 hours: pleased when it is up, worried when it is down. The cover photo shows one at work (its screen names and number of dots may differ slightly from what yours shows).

What it does

- Fetches prices straight from public data providers (Yahoo Finance, CoinGecko, CNN and alternative.me for the Fear & Greed indexes). There is no Figo server and no account.
- Rotates through your screens on its own, every 8 seconds. You can also swipe the glass to move.
- Shows a face that follows one "starred" screen.
- Dims and falls asleep at night, if you want it to.
- Is set up and controlled entirely from a web page on your phone. There is no app to install.



Render: the Bullish face.



Render: a Fear & Greed screen.

What it is made of

Figo is one ready-made board — the Waveshare ESP32-C6-Touch-LCD-1.47, a WiFi microcontroller with a 1.47-inch touch screen already attached — inside a 3D-printed stand. There is no soldering and no wiring.

How this guide is organised

CHAPTERS	YOU WILL
2–3	Buy the board and print the stand.
4–8	Install the Arduino IDE, set it up, and put the firmware on the board.

CHAPTERS	YOU WILL
9	Fit the board into the stand.
10–13	Switch Figo on, connect it to WiFi and make it yours.
14–15	Fix problems, or start again from scratch.
16	Reference: the web API and the settings format.

> **Note** Photos marked *render* are computer images of the design, and panel pictures are the design files the firmware was built to match. Phone screenshots are of the real phone page.

CHAPTER 2

What you need

Parts

ITEM	NOTES
Waveshare ESP32-C6-Touch-LCD-1.47	The board with the **touch** screen. See "Get the right board" below.
USB-C cable	Must carry **data**, not just power. A charge-only cable powers the board but the computer never sees it.
USB power	A phone charger or any computer USB port.
The printed stand	One part, printed in PLA. Chapter 3.

Tools and accounts

- A computer that runs **Arduino IDE 2.x**. Everything in this guide was done on Windows; macOS and Linux should work the same way but were not tried.
- A phone with a web browser. It does not need an app.
- A **2.4 GHz** WiFi network. The ESP32-C6 cannot see 5 GHz networks. Most home routers broadcast both; if yours has separate names for each band, use the 2.4 GHz one.
- Access to a 3D printer (your own, a friend's, or a print service).

No screws, glue or other parts are needed: the board press-fits into the stand. No accounts or API keys are needed either. Every data provider Figo uses works without one.

Get the right board

Waveshare sells more than one board that looks like this. The one Figo is written for has these chips:

PART	CHIP ON THE FIGO BOARD	NOT THIS (SIBLING BOARD)
Display	JD9853, 172 × 320	ST7789
Touch	AXS5106L	CST816S
Processor	ESP32-C6 with **8 MB** flash	—

Check the product page's specification table for the display and touch chip names before you buy. Some older write-ups of this project named the ST7789/CST816S chips; that was a mistake.

> **Why** The firmware drives these exact display and touch chips. It has never been run on the sibling board and is not expected to work there.

CHAPTER 3

Printing the stand

The stand is one part. Its print file is `design/Figo.3mf` in the repository.



Render: the stand. The board sits behind the front bezel; the USB-C port lines up with the opening on the right.

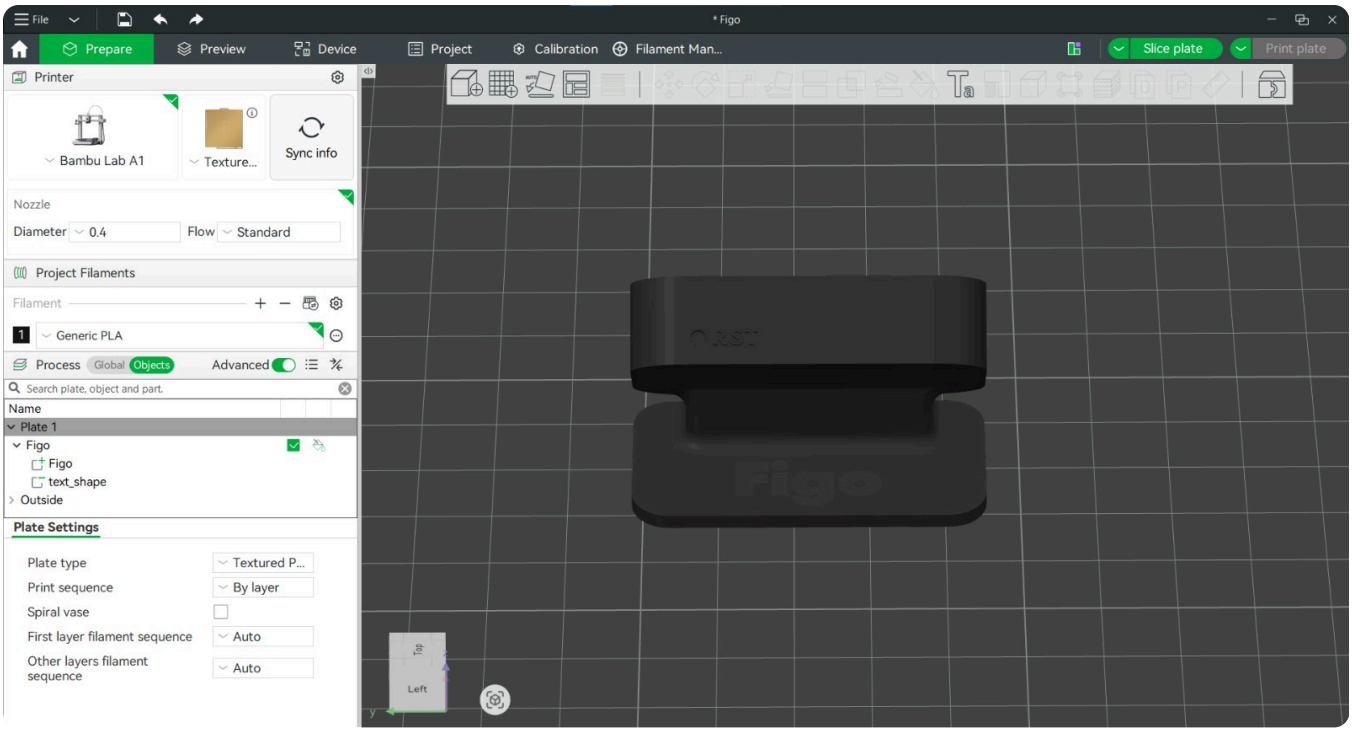
The print settings the fitted part used

`Figo.3mf` is a Bambu Studio project. The stand that holds the author's board was printed from it with these settings, read straight out of the file:

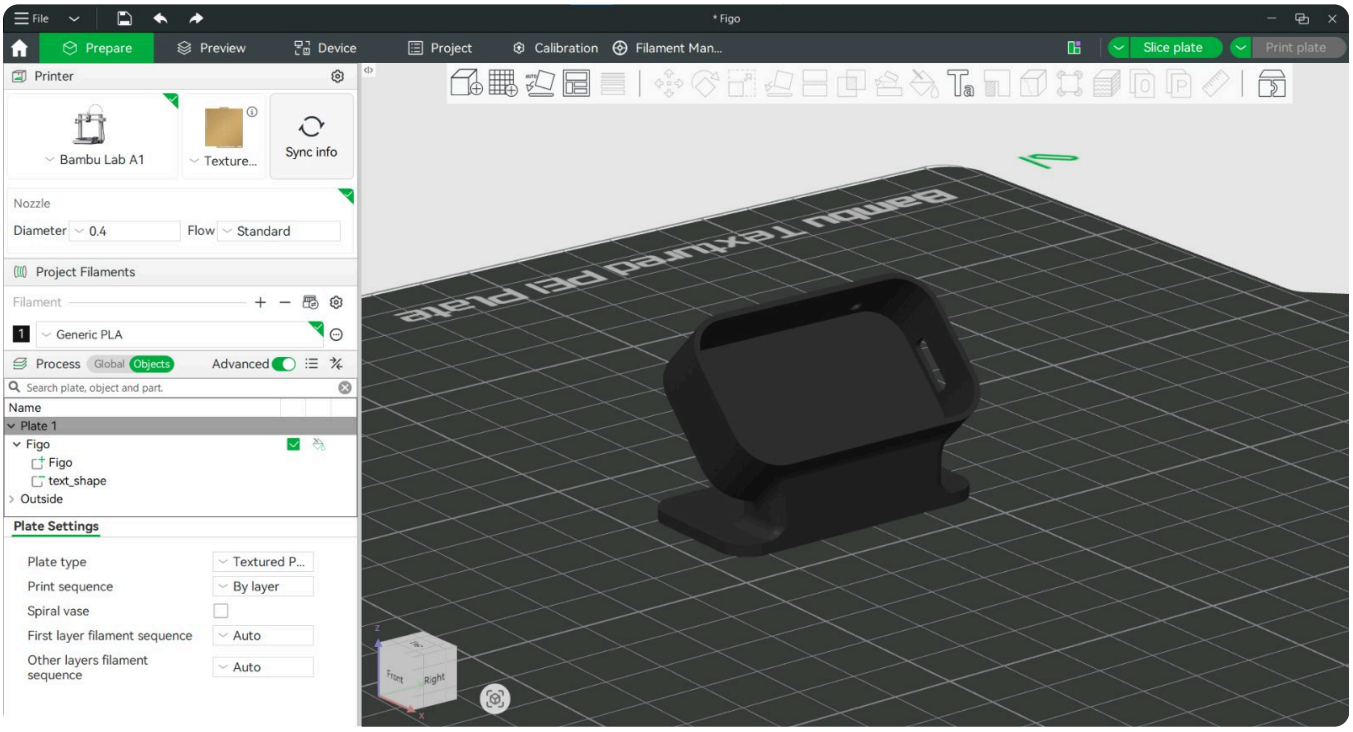
SETTING	VALUE
Printer	Bambu Lab A1, 0.4 mm nozzle
Process profile	0.20 mm Standard
Material	PLA (Generic PLA profile)
Nozzle temperature	220 °C
Bed	Textured PEI plate, 65 °C
Layer height	0.20 mm (first layer 0.20 mm)
Walls	2
Top / bottom layers	5 / 3
Infill	15 %, grid
Supports	**Off**
Brim	Auto
Seam	Aligned

SETTING	VALUE
Orientation	Standing on its base, the way it sits on a desk

The word "Figo" is embossed into the base as part of the model. You do not need to add it.



Bambu Studio, front view. The printer and filament settings are in the left-hand panel.



On the plate: standing on its base, no supports.

Printing it

1. **Bambu Studio users:** open `design/Figo.3mf`. Everything above is already set. Slice and print.

2. **OrcaSlicer users:** open the same file. Orca reads Bambu projects, including their settings. Check the table above against what it loaded, then slice and print.
3. **Any other slicer (PrusaSlicer, Cura, ...):** import `design/Figo.3mf`. You get the shape only, not the settings. Keep the orientation it opens in, enter the settings from the table, and turn supports off.

> **Note** The stand was designed to be printed without supports. If your slicer warns about overhangs, check its preview before adding any: support material inside the bezel is hard to remove and can stop the board from seating.

Before you go further

The stand holds the screen behind a bezel, and it is **not designed to come apart again and again**. So the board gets its firmware *before* it goes in (chapter 8), and only then is it assembled (chapter 9). Put the printed stand aside until then.

CHAPTER 4

Arduino IDE and the ESP32 board package

Figo's firmware is built and uploaded with the Arduino IDE. This chapter installs it and teaches it about the ESP32-C6.

Install the IDE

1. Download **Arduino IDE 2.x** from arduino.cc and install it. (Version 1.8 is a different program; this guide does not cover it.)
2. Start it once so it can finish its first-run setup.

Add the ESP32 board package

1. Open **File** → **Preferences** (on macOS: **Arduino IDE** → **Settings**).
2. In **Additional boards manager URLs**, add: `https://espressif.github.io/arduino-esp32/package_esp32_index.json`
3. Click **OK**.
4. Open **Tools** → **Board** → **Boards Manager**.
5. Search for **esp32** and install **esp32 by Espressif Systems**, version **3.3.11**.

> **Why this version** Figo was built and tested on 3.3.11. It needs a 3.x version: the 2.x packages use a different backlight (LEDC) API and an older network stack, and the firmware will not compile against them. Newer 3.x versions will probably work but were not tested.

Get the code

Figo is published as a public repository on GitHub: **https://github.com/NurSharlin/Figo**. The firmware lives in its `Figo` folder.

- **With git:** `git clone https://github.com/NurSharlin/Figo.git`
- **Without git:** on the GitHub page, choose **Code** → **Download ZIP** and unzip it.

Either way you end up with a folder that contains, among other things:

```
Figo/                 <- the sketch folder
  Figo.ino            <- the main sketch
  lv_conf.h
  build_opt.h
  ... about 40 more .cpp and .h files
design/Figo.3mf       <- the stand (chapter 3)
docs/guide/           <- this guide and the quick reference, as PDFs
```

Chapter 7 explains why these files must stay exactly where they are.

CHAPTER 5

Library versions that work

Figo needs five libraries. Install each one from **Tools → Manage Libraries…** (the Library Manager). Search for the name in the first column and install the exact version shown.

LIBRARY (AS NAMED IN THE LIBRARY MANAGER)	AUTHOR	VERSION	WHAT GOES WRONG WITH THE WRONG ONE
lvgl	kisvegabor	**8.4.0**	**Not 9.x.** LVGL 9 renamed most of the API the firmware uses and changed the format of `lv_conf.h`. The build stops with hundreds of errors about undeclared `lv_…` names.
ArduinoJson	Benoit Blanchon	**7.x** (tested: 7.4.3)	Version 6 has no `JsonDocument` class the way the firmware uses it; the build stops with errors about `JsonDocument`.
ESP Async WebServer	ESP32Async	**3.12.0**	Use the **ESP32Async** one. The older `me-no-dev` library (and copies of it) was archived in January 2025, does not build against the 3.x ESP32 package, and lacks the `AsyncURIMatcher` the firmware uses.
Async TCP	ESP32Async	**3.5.0**	Same rule: the **ESP32Async** one. `ESPAsyncTCP` (without the space, by dvarrel and others) is for the older ESP8266 chip and is not what you want.
GFX Library for Arduino	Moon On Our Nation	**1.6.7**	Draws the pixels on the panel. Tested with 1.6.7 only; other versions were not tried.

> **Warning** When the Library Manager asks whether to install a library's dependencies too, read the list. If it offers an ESP8266 library or a `me-no-dev` library, choose **install without dependencies** and install the ones above yourself.

Nothing else is needed. In particular Figo does **not** use LittleFS, SPIFFS or a WiFi-manager library; the phone page is built into the firmware.

Checking what you have

Tools → Manage Libraries…, then set the **Type** filter to *Installed*. Each library shows its installed version, and a drop-down lets you switch to another version.

CHAPTER 6

The two settings everyone gets wrong

Open `Figo/Figo.ino` (**File** → **Open...**), plug the board in with the USB-C cable, and set the **Tools** menu like this:

TOOLS MENU ENTRY	SET IT TO
Board	**ESP32C6 Dev Module** (under *esp32*)
Port	The port that appears when you plug the board in (on Windows: a `COM` port)
Flash Size	**8MB (64Mb)**
Partition Scheme	**8M with spiffs (3MB APP/1.5MB SPIFFS)**
USB CDC On Boot	**Enabled**

Leave every other entry at its default.

Setting 1: Flash Size and Partition Scheme

The board has 8 MB of flash, but the IDE assumes 4 MB unless told otherwise. Set **both** entries — they go together.

> **Warning Symptom when wrong:** the build runs for a long time and then fails at the very end with `text section exceeds available space in board` and a figure like **127%**. It looks like the firmware is too big. It is not: the default scheme gives it only 1.2 MB, and it needs about 2.1 MB. Fix the two settings and build again.

Setting only the partition scheme and leaving Flash Size at 4 MB is also wrong: the board's startup code then describes a 4 MB chip to an 8 MB layout.

> **Why** The 8 MB scheme has two 3.2 MB slots for the firmware. The second slot is what lets you update Figo over WiFi later (chapter 12). The 1.5 MB "spiffs" part is reserved and unused.

Setting 2: USB CDC On Boot

> **Warning Symptom when wrong:** Figo works perfectly, but the **Serial Monitor shows nothing** (apart from a few lines of boot text from the chip itself). Every diagnostic message the firmware prints goes to pins on the board instead of the USB cable.

You only notice this when something goes wrong and you need the log — which is exactly when you least want to rebuild. Set it now.

If you build from the command line

The same three settings, in `arduino-cli` form:

```
arduino-cli compile --fqbn
"esp32:esp32:esp32c6:FlashSize=8M,PartitionScheme=default_8MB,CDCOnBoot=cdc" Figo
```

The IDE's menu choices are not shared with `arduino-cli`; `CDCOnBoot=cdc` in particular must be written out, because the command-line default is off. The IDE also ships `arduino-cli` with an empty configuration, so add `--libraries "<your sketchbook>/libraries"` or it will not find the libraries from chapter 5.

CHAPTER 7

File placement

Three rules. Break any one and the build fails before it gets anywhere interesting.

1. The folder and the sketch share a name

The Arduino IDE only opens a sketch whose file name matches its folder: `Figo/Figo.ino`. If you downloaded a ZIP, the unzipped folder may be called something like `Figo-main`. The *repository* folder can have any name; the **sketch** folder inside it must be called exactly `Figo`.

All the `.cpp` and `.h` files sit **next to** `Figo.ino`, in the same folder. Do not move them into sub-folders: the IDE only compiles files at the top of the sketch folder (and in a folder called `src`, which Figo does not use).

2. `lv_conf.h` stays in the sketch folder

`lv_conf.h` is LVGL's settings file. Figo's copy is set up for this panel: 320 × 172, 16-bit colour, and the exact fonts the screens use. It lives in `Figo/`, next to `Figo.ino`.

Do **not** copy it into the LVGL library folder, and do not use the example `lv_conf.h` that comes with LVGL.

3. `build_opt.h` stays next to it

`build_opt.h` is one line long:

```
-DLV_CONF_INCLUDE_SIMPLE
```

It tells LVGL to look for `lv_conf.h` by its plain name, which is what lets the library (installed in your sketchbook) find the settings file (in the sketch folder).

> **Warning Symptom when missing:** LVGL cannot find its settings file. Depending on the version you see either an error about `lv_conf.h`, or LVGL builds with its own defaults and the fonts the screens ask for do not exist.

The phone page is already inside

The web page your phone opens is also part of the firmware. Its readable source is in `Figo/data/` and it has already been packed into `Figo/WebAssets.h`, which is in the repository. **There is nothing to upload separately** — no filesystem image, no "data upload" tool.

If you change anything in `Figo/data/`, run this from the repository folder before building (it needs Node.js):

```
node Figo/tools/embed-spa/embed.mjs
```

If you skip it, the firmware quietly keeps serving the old page.

CHAPTER 8

Flashing

"Flashing" means building the firmware and writing it onto the board. Do it now, with the board still loose on the desk.

> **Why before assembly** Once the board is in the stand its **BOOT button cannot be reached**, and BOOT is what rescues a first upload that will not connect. The stand is also a press fit that is much easier to put a board into than to get one out of. So get Figo booting on the desk first, and only then put him in his case.

Build and upload

1. Plug the board into the computer with the USB-C **data** cable.
2. Check the **Tools** menu once more against chapter 6: board, port, Flash Size, Partition Scheme, USB CDC On Boot.
3. Click **Upload** (the right-arrow button).
4. Wait. The **first build takes roughly 15 to 25 minutes**, most of it compiling LVGL, and the IDE shows very little while it works. It has not hung. Later builds reuse that work and take about a minute.
5. When the build ends, the IDE uploads the firmware. You see a line like `Sketch uses 2,1xx,xxx bytes (6x%) of program storage space` and then the upload progress.
6. The board restarts on its own when the upload finishes.

What you should see

The screen shows a greeting for a couple of seconds:



Hello — shown every time Figo starts.

Then, because the board has no WiFi details yet, it asks you to join its setup network:



Figo is ready to be set up. You will use this in chapter 10.

If you see this, the firmware is on the board and working. Unplug it and go to chapter 9.

Optional: look at the log

With the board plugged in, open **Tools** → **Serial Monitor** and set the speed to **115200 baud**. Press the **RESET** button on the top edge of the board to see Figo start up again. If this window stays empty, "USB CDC On Boot" is not enabled (chapter 6).

If the upload fails

- **No port in the Tools → Port menu:** the cable is probably charge-only. Try another.
- **"Failed to connect" or "No serial data received":** put the board into download mode by hand. Hold **BOOT**, press and release **RESET** (on the top edge), then release **BOOT**, and click Upload again. After a manual download-mode upload, press RESET once to start the new firmware. This only works while the board is out of the stand: in the stand, BOOT cannot be reached.
- **The build fails at the end with "127%":** Flash Size and Partition Scheme (chapter 6).
- **Hundreds of errors about `lv_…` names:** wrong LVGL version (chapter 5).

CHAPTER 9

Assembling the board into the stand

Only do this once the board shows the Connect screen from chapter 8.



A seated board. The USB-C cable leaves through the opening on the right-hand side.

Which way round

Look at the stand from the front. The opening for the USB-C port is on the **right-hand side**. The board goes in with:

- the screen facing out, towards you, and
- the USB-C port lined up with that side opening.

This is the orientation the firmware calls **Normal — cable on the right**, and it is the default. (If you ever mount a board the other way up, chapter 12 shows the setting that flips the picture.)

Fitting it

1. Unplug the USB cable.
2. Hold the board with the screen facing you and the USB-C port towards the side opening.
3. Press it into the front of the stand until it is seated. It is a **press fit**: nothing else holds it — no screws, no glue.
4. Check from the front that the whole screen is visible inside the bezel and nothing is pressing on the glass.
5. Plug the USB-C cable back in through the side opening.

The screen lights up with the hello screen, then the Connect screen, exactly as it did on the desk. The small hole marked **RST** on the stand lines up with the board's RESET button: press through it with something thin to restart Figo without unplugging it.

> **Warning** The stand captures the panel and is **not designed to come apart repeatedly**. To take the board out, lift it by the glass with a **suction cup (vacuum lifter)** — the kind sold for phone-screen repairs. Do not lever against the glass. Once it is in the stand, the board's BOOT button cannot be reached, which is why chapter 8 comes first.

CHAPTER 10

First switch-on and joining WiFi

Figo has no keyboard, so it borrows your phone. It opens a small WiFi network of its own, your phone joins it, and a setup page tells Figo which home network to use.

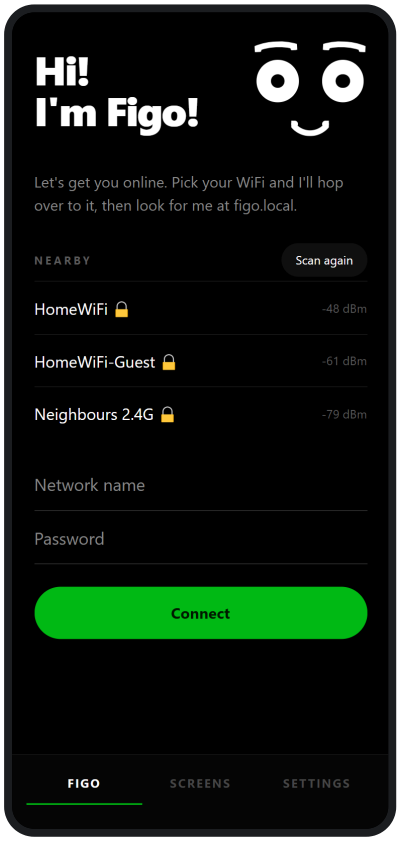
Connect Figo to your WiFi

1. Plug Figo in. After the greeting, the screen shows the setup network's name and password.

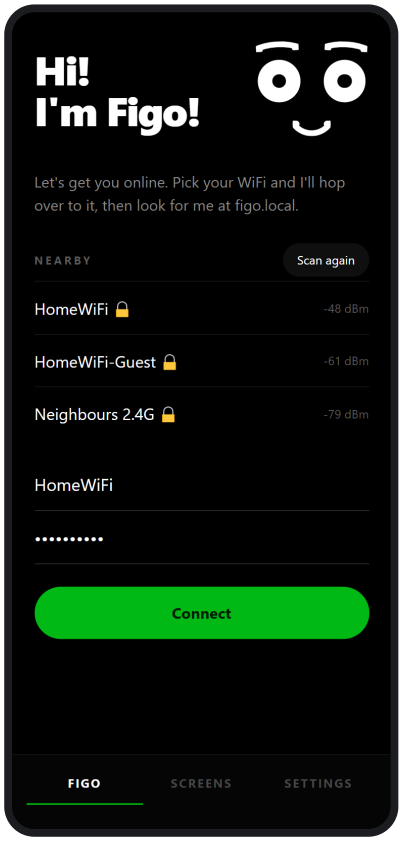


Network: Figo-Setup. Password: figo1234.

2. On your phone, open the WiFi settings and join **Figo-Setup** with the password **figo1234**.
3. Most phones then open the setup page by themselves. If yours does not, open a browser and go to **http://192.168.4.1**.
4. Tap your home network in the list (or type its name if it is hidden), then type its password.



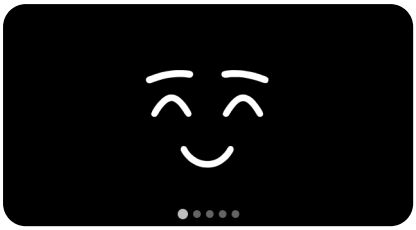
The setup page lists the networks Figo can see.



Network picked, password typed.

5. Tap **Connect**. It can take several seconds. When it works, the page says **Connected! Find me at figo.local**.

6. A few seconds later the Figo-Setup network disappears. Most phones go back to your home WiFi by themselves; if yours does not, switch it back.
7. On Figo's screen, the carousel starts: the face first, then the prices.



The face comes first.



Then the price screens, one every 8 seconds. (Design picture: yours starts with the S&P 500.)

> **Note** Figo can only join **2.4 GHz** networks. If your network is not in the list, check that its 2.4 GHz band is switched on.

If Figo cannot join — a mistyped password, say — the page shows **Could not connect to that network** and you can try again. Nothing is saved until a connection actually works.

Open the phone page

From now on, Figo's control page is at:

http://figo.local

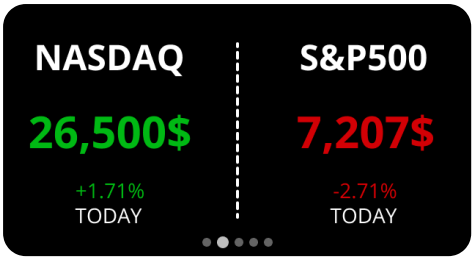
Open it on any phone or computer on the same WiFi. Bookmark it, or use your browser's **Add to Home Screen** so it opens like an app.

> **Tip** If `figo.local` does not open (some Android phones do not support `.local` names), find Figo's address in your router's list of connected devices — it shows up as an Espressif device — and open that address instead, for example `http://192.168.1.42`. Once you are in, **Settings** shows the address too.

What the carousel shows

A new Figo comes with four screens, plus the face:

#	SCREEN	SHOWS	REFRESHES
—	Face	Figo's mood (chapter 13)	—
1	S&P 500	The US stock index, one price	every 15 min
2	BTC · ETH	Bitcoin and Ethereum side by side. **Starred**: the face follows Bitcoin	every 2 min
3	Gold	Gold futures	every 15 min
4	Fear & Greed	CNN's stock-market Fear & Greed index, as a round gauge	every 15 min



A dual screen: two prices side by side. (Design picture; yours shows BTC and ETH.)



The Fear & Greed gauge.

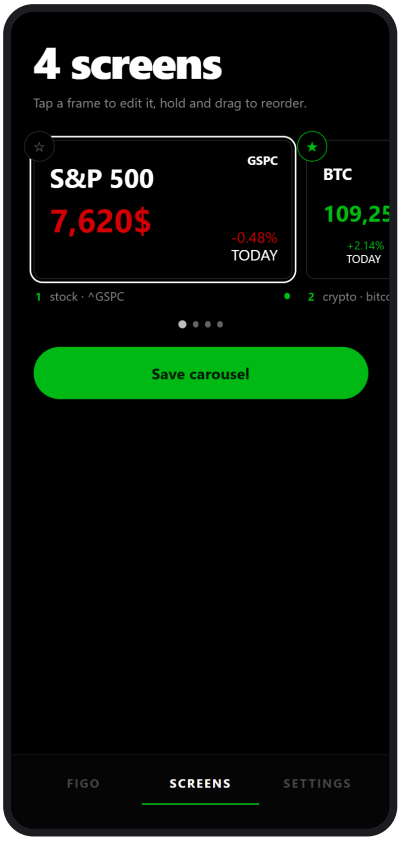
Figo moves to the next screen every 8 seconds. Swipe the glass left or right to move yourself; after a swipe it waits the full 8 seconds before moving on. The dots along the bottom show where you are.

Prices appear as soon as each screen has fetched once — usually within a minute or two of switching on. A screen that has not fetched yet shows `--`.

CHAPTER 11

Setting up screens from your phone

Open **http://figo.local** and tap **SCREENS** at the bottom.



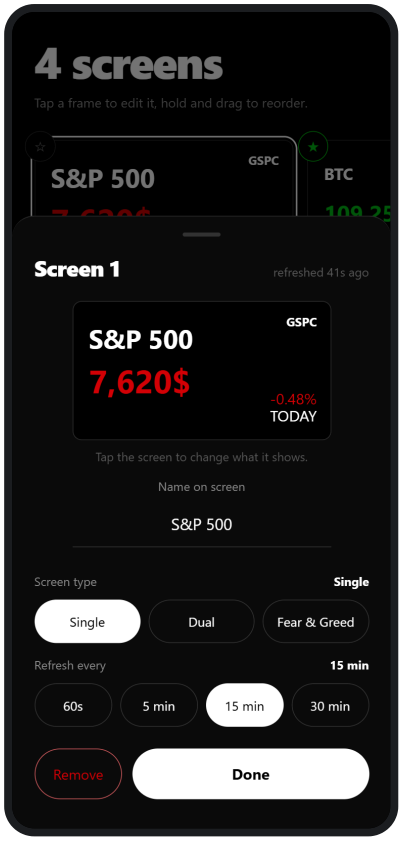
Each frame is a copy of one of Figo's screens, with live prices. The star marks the screen the face follows.

The strip of frames is your carousel, in order. Swipe it sideways to see them all. Under each frame: its position, what kind of screen it is, and a dot — green when its data is fresh.

> **Note** Changes on this tab stay on your phone until you tap **Save carousel**. Only then does Figo's screen change.

Change a screen

1. Tap a frame. An editor slides up with a larger copy of the screen.

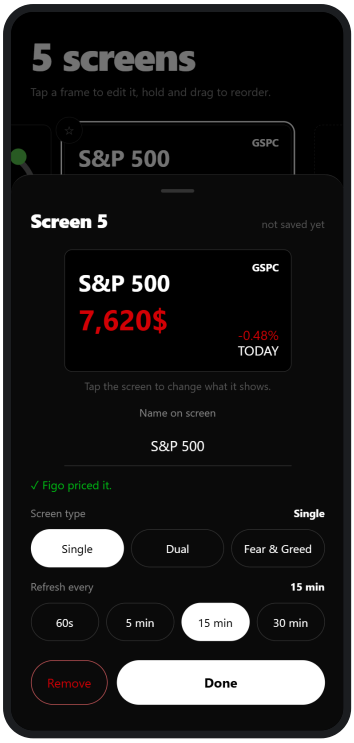


The editor. The preview is itself the control.

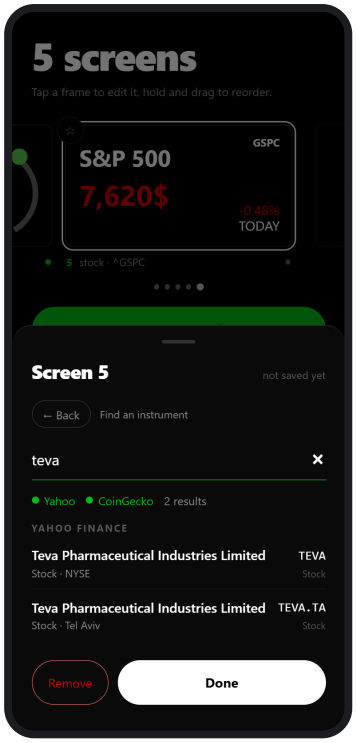
2. To change what it shows, **tap the preview**. On a dual screen, tap the half you want to change.
3. **Name on screen** is the title Figo displays. Up to 12 characters (10 for each half of a dual screen).
4. **Screen type**: *Single* (one price), *Dual* (two prices side by side) or *Fear & Greed* (a gauge).
5. **Refresh every**: how often Figo fetches the price. The shortest is 60 seconds (2 minutes for a dual screen, which fetches two prices).
6. Tap **Done**, then **Save carousel**.

Add a screen

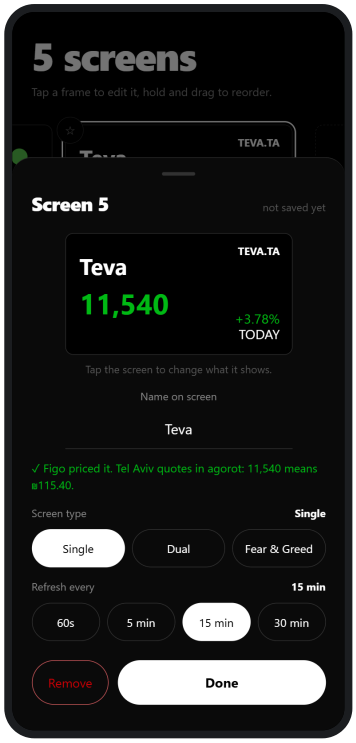
1. Tap **+ Add** at the right-hand end of the strip. A new screen opens, already set to something that works.
2. Tap the preview to search. **Type a name, not a ticker symbol** — "Teva", "Nasdaq", "Solana", "Gold". Figo searches Yahoo Finance and CoinGecko and lists what both know.
3. Tap the result you mean. Figo fetches one live price to prove it can, fills in a short name, and says **Figo priced it**.



1 · A new screen.



2 · Searching by name.



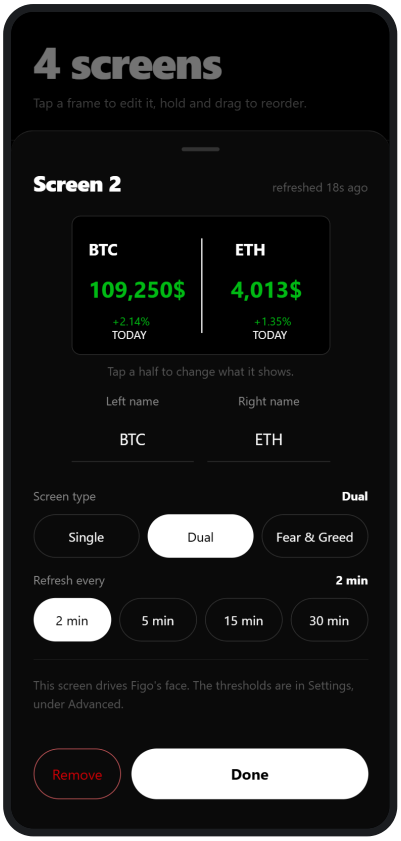
3 · Checked. Tel Aviv prices are in agorot, and it says so.

4 Change the name if you like, tap **Done**, then **Save carousel**.

Figo holds up to **8** screens. When the strip is full, the **+ Add** button says **Max 8**.

Reorder, remove, star

- **Reorder:** press and hold a frame until it lifts, then drag it along the strip. Let go where you want it. Save carousel.
- **Remove:** open the frame, tap **Remove**. Save carousel.
- **Star:** tap the star above a frame. The face now follows that screen (chapter 13). A screen is always starred; to move the star, tap another one. Save carousel.



The starred screen's editor says so at the bottom.

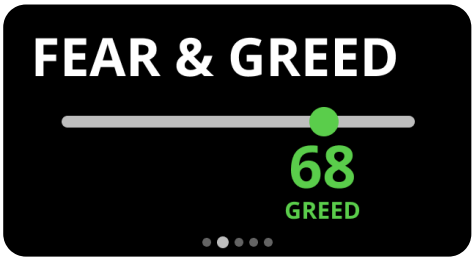
Fear & Greed screens

Choose **Fear & Greed** as the screen type, then:

- **Index:** *Stock market* (CNN's Fear & Greed index) or *Crypto* (alternative.me's Crypto Fear & Greed index).
- **Gauge:** *Circular* (a round dial) or *Slider* (a straight bar).



Round gauge.



Slider.

What the prices mean

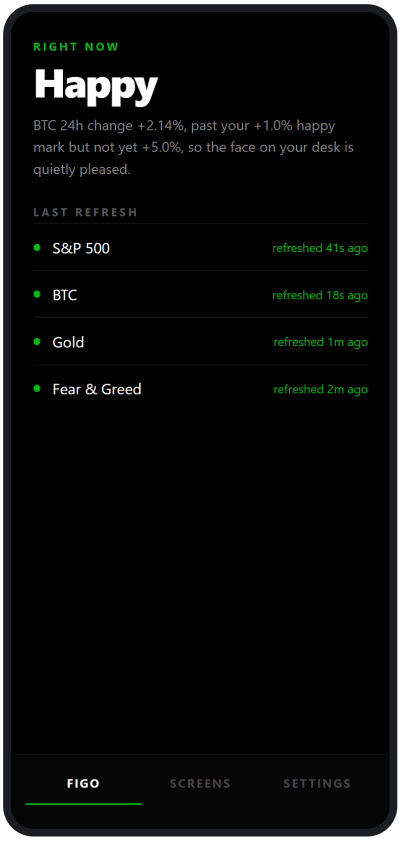
- Green means up on the day, red means down.
- Prices are shown in the currency the exchange quotes. Tel Aviv shares are quoted in agorot (1/100 of a shekel).
- If a screen cannot get new data for a while, its price turns grey and an error code appears in place of the change. Chapter 14 lists the codes.

CHAPTER 12

Settings, faces and the star

The FIGO tab

The first tab tells you what the face on your desk is doing, and why.

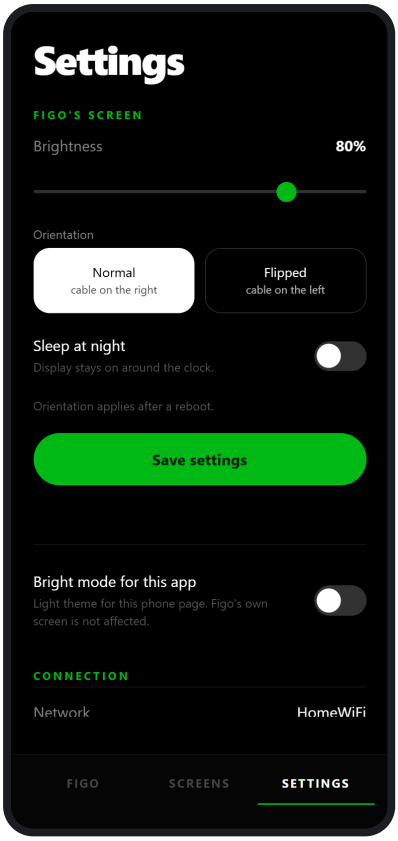


Right now: Happy, and the reason in plain words. Below, how recently each screen fetched its data.

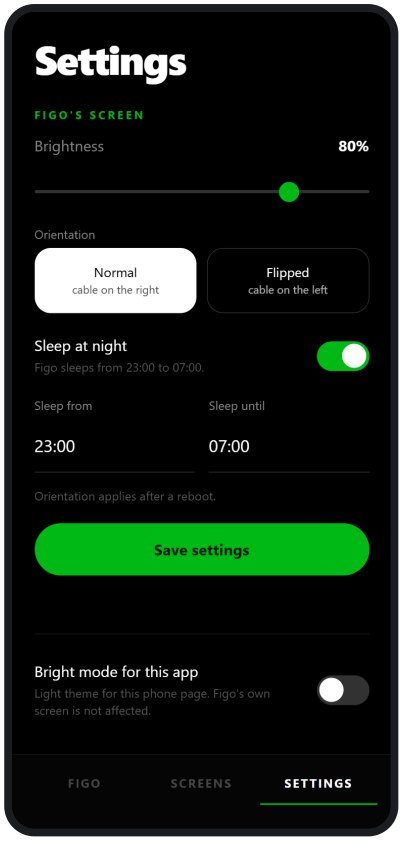
Each screen has a coloured dot: **green** when its data is fresh, **amber** when it has stopped updating.

Settings

Tap **SETTINGS**. Changes here take effect when you tap **Save settings**.



Figo's screen: brightness, orientation and sleep.



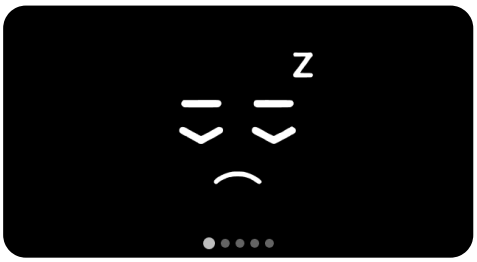
Sleep at night, 23:00 to 07:00.

Brightness — from 10 % to 100 %. It cannot go below 10 %: a dark screen looks exactly like a broken one.

Orientation — *Normal* (cable on the right) or *Flipped* (cable on the left), for a board mounted the other way up. This one needs a restart: Figo applies it the next time it starts. (Use **Reboot**, below.)

Sleep at night — switch it on and choose *Sleep from* and *Sleep until*, in 15-minute steps. During that window:

- the screen dims to 5 % and shows the sleeping face;
- touching the glass wakes Figo up for the rest of that night's window;
- at the end of the window Figo wakes by itself and shows the current face straight away.



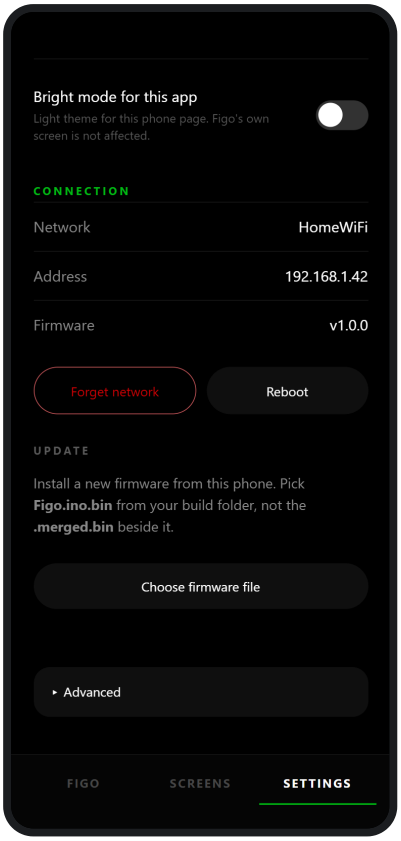
Asleep.

Figo gets the time from the internet and your time zone from your phone, each time you save settings. After a daylight-saving change, just open the phone page once: it notices that the phone's clock and Figo's disagree and corrects Figo on its own.

Bright mode for this app — a light theme for the phone page only. Figo's own screen is not affected. This one is saved on the phone straight away.

Connection and updates

Further down the Settings tab:



Network, address and firmware version, then the Update card.

- **Network / Address / Firmware** — the WiFi Figo is on, its address, and the firmware version.
- **Forget network** — erases the saved WiFi and restarts Figo into setup (chapter 10). Your screens and settings are kept.
- **Reboot** — restarts Figo.

Updating the firmware over WiFi

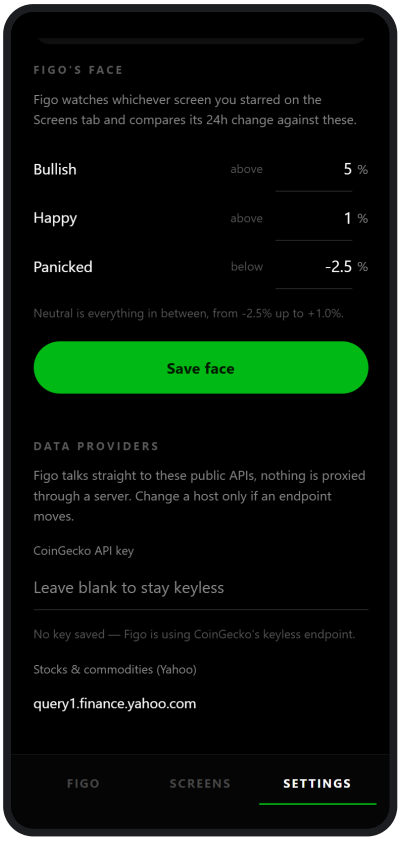
After the first USB upload, new firmware can go on without a cable.

1. In the Arduino IDE, choose **Sketch → Export Compiled Binary**. The IDE builds Figo and saves the result in a `build` folder inside the sketch folder.
2. Copy `Figo.ino.bin` from that folder to your phone. Not `Figo.ino.merged.bin` — Figo refuses that one.
3. On the phone page, **Settings → Update → Choose firmware file**, and pick it. The page reads the version out of the file and asks before installing.
4. Tap **Install**, keep the page open and Figo plugged in. The screen shows the progress, then Figo restarts on the new version.

If a new version does not start up properly, Figo goes back to the previous one by itself within about two minutes. If a new version runs but its Update card is broken, **http://figo.local/update** is a plain stand-alone update page.

Advanced: the face, the providers, starting over

Tap **Advanced** at the bottom of Settings.



The three thresholds that decide Figo's face, then the data providers.

Figo's face — three numbers, in percent change over 24 hours on the starred screen. Chapter 13 explains what they do. Each must be higher than the one below it. Tap **Save face**.

Data providers — the four web addresses Figo fetches from, and an optional CoinGecko API key. Leave them alone unless a provider moves its address. Hostnames only: no `https://`, no port, no path. Tap **Save providers**.

Start over — erases everything. Chapter 15.

The star

The star on the Screens tab (chapter 11) picks **which screen the face follows**. A new Figo stars the Bitcoin screen.

> **Why Bitcoin** Crypto trades around the clock. A stock index stops moving every evening and weekend, so a face following one would sit still for most of the week.

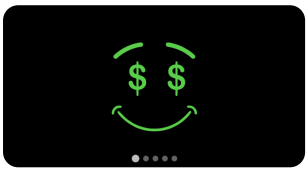
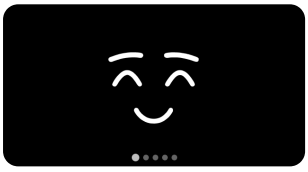
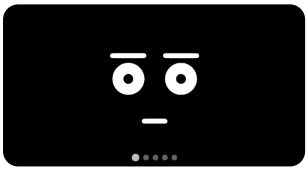
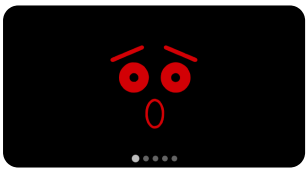
On a dual screen, the face follows the **left-hand** instrument.

CHAPTER 13

What each face means

The face is the first screen in the carousel. It shows how the **starred** screen's price has moved over the last 24 hours, sorted into one of four moods by three thresholds you can change (Settings → Advanced → Figo's face).

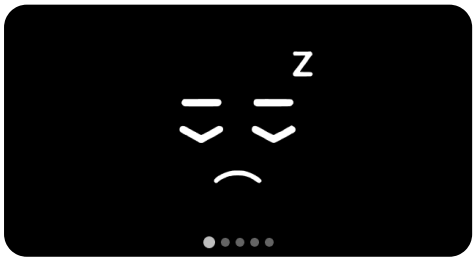
The four moods

FACE	MOOD	WHEN (FACTORY THRESHOLDS)
	**Bullish**	Up **5 %** or more
	**Happy**	Up **1 %** or more, but less than 5 %
	**Neutral**	Anything between **−2.5 %** and +1 %
	**Panicked**	Down **2.5 %** or more

Neutral has no threshold of its own: it is whatever is left between *Happy* and *Panicked*.

The factory thresholds suit Bitcoin, which moves a lot in a day. If you star a calmer instrument — a stock index rarely moves 5 % in a day — lower them, or the face will hardly ever leave Neutral.

Two more faces



Sleeping — during the night window, if you set one (chapter 12).



Hello — when Figo starts up. The same face appears on the screen that shows a firmware update installing.

When Figo does not know

If there is nothing to go on, the face shows **Neutral** rather than guessing. That happens when:

- the starred screen has not fetched a price yet (just after switching on);
- the starred screen's data has gone stale — it has failed to update several times in a row;
- the starred screen was removed.

The FIGO tab on the phone says which of these it is, where the panel has no room to.

CHAPTER 14

Troubleshooting

Building and uploading

WHAT YOU SEE	WHAT TO DO
Build fails at the end: `text section exceeds available space in board`**, 127%**	Flash Size **8MB** and Partition Scheme **8M with spiffs** (chapter 6).
Hundreds of errors about `lv_…` **names**	LVGL must be **8.4.0**, not 9.x (chapter 5).
Errors about `JsonDocument`	ArduinoJson must be **7.x** (chapter 5).
Errors about `AsyncURIMatcher` **or** `AsyncWebServer`	Wrong web-server library: use the **ESP32Async** ones (chapter 5).
`lv_conf.h` **not found, or missing fonts**	`lv_conf.h` and `build_opt.h` must sit next to `Figo.ino` (chapter 7).
The IDE will not open the sketch	The sketch folder must be called `Figo` (chapter 7).
No port in Tools → Port	Charge-only cable. Try another.
"Failed to connect" during upload	Download mode by hand: hold BOOT, tap RESET, release BOOT (chapter 8). Only possible with the board out of the stand.
Serial Monitor is empty	USB CDC On Boot → **Enabled** (chapter 6), and the monitor at **115200** baud.
The build seems stuck	The first build takes 15–25 minutes. Give it time.

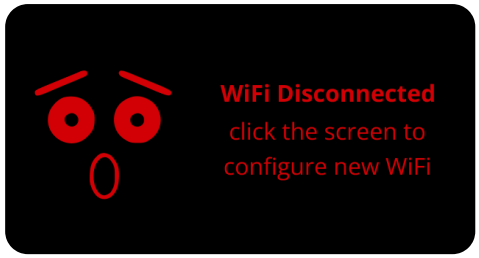
WiFi

The screen stays on "Connect to my WiFi for setup". Figo has no saved WiFi, or cannot use it. Join **Figo-Setup** from your phone and set it up again (chapter 10).

Figo-Setup opens, but the setup page does not. Browse to **http://192.168.4.1** while still on Figo-Setup.

My network is not in the list. Figo only sees **2.4 GHz** networks. Tap **Scan again**, or type the network name by hand.

"WiFi Disconnected" on the screen



Shown when a working WiFi connection has been down for about 20 seconds.

Figo lost its WiFi. It keeps showing prices for the first 20 seconds, then shows this screen, and tries the saved network again **every minute by itself** — after a router restart you do not need to do anything. If your WiFi has actually changed (new router, new password), **tap the screen**: Figo opens its Figo-Setup network so you can set it up again (chapter 10).

`figo.local` **does not open.** Some Android phones do not support `.local` names. Use Figo's address instead — it is under Settings → Connection, and in your router's list of connected devices.

Prices and the face

A screen shows `--`. It has not fetched yet. Wait a minute or two after switching on.

A price has turned grey, with "Error" and a number. That screen has failed to update three times in a row. A grey price is the last one Figo got; the number says why (table below). Figo keeps retrying, waiting longer between tries (up to 5 minutes), and the screen goes back to normal on its own as soon as a fetch works. If a screen has *never* worked, the error code appears in place of the price.

The face is always Neutral. Either the starred screen has no fresh data (open the FIGO tab: it says why), or the thresholds are too wide for what you starred (chapter 13).

Figo sleeps at the wrong time. Open the phone page once: it corrects Figo's time zone from your phone. Then check the times in Settings.

The error codes

These are the numbers Figo can show. The table is generated from the firmware's own list, so it always matches the version this guide was built with.

CODE	WHERE	WHAT IT MEANS
40	Network	WiFi dropped. Not shown on the panel (the WiFi Disconnected screen is), only written to the serial log.
50	Screen	The data provider did not answer, or answered with an error.
51	Screen	The provider answered, but not with the data Figo asked for.
52	Screen	CoinGecko does not know this coin. Pick it again with the search box.
53	Screen	The provider is limiting requests (HTTP 429). Figo backs off and retries on its own.
54	Screen	A number came back but cannot be right (a price of zero, a Fear & Greed value outside 0 to 100).

CODE	WHERE	WHAT IT MEANS
60	Boot	Saved WiFi settings could not be read. Figo started without them.
61	Boot	Saved brightness / orientation / sleep settings could not be read. Defaults are in use.
62	Boot	Saved face settings could not be read. Defaults are in use.
63	Boot	Saved provider hosts could not be read. Defaults are in use.
64	Boot	Saved screens could not be read. The factory screen set is in use.

- **Screen** codes (5x) replace the change figure on a screen that has failed three times in a row.
- **Boot** codes (6x) appear on a screen shown for about five seconds at start-up, with a red face and the words **Settings problem — Using defaults**, for example `Error 61 64`. Part of Figo's saved settings could not be read, so Figo carries on with factory values for that part. Open the phone page and set that part again; saving it overwrites the damaged copy. If it keeps coming back, a factory reset (chapter 15) clears everything.
- **Network** code 40 is only written to the serial log. On the panel, a lost connection shows the WiFi Disconnected screen above instead.

Updates

The phone says the update failed. The reason is on the phone and on Figo's screen. Figo keeps running the version it had. The most common cause is picking `Figo.ino.merged.bin`; pick `Figo.ino.bin`.

Figo went back to the old version after an update. The new one did not start up properly within about two minutes, so Figo reverted to the last good one on its own. Connect the USB cable and look at the Serial Monitor while it starts to see why.

An update broke the updater. A bug in the Update card itself can be worked round at **http://figo.local/update**. A bug that stops Figo starting at all needs a USB upload (chapter 8). The cable reaches the board through the side of the stand, so try that first. If the upload will not connect, the board has to come out of the stand (chapter 9) so you can reach BOOT.

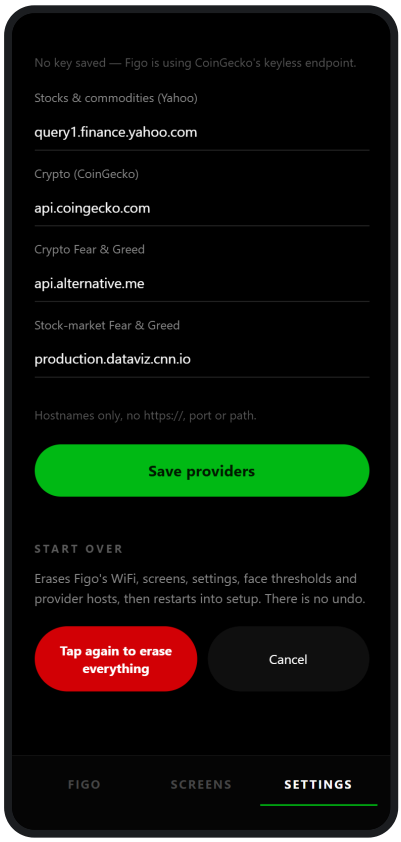
CHAPTER 15

Putting Figo back to factory settings

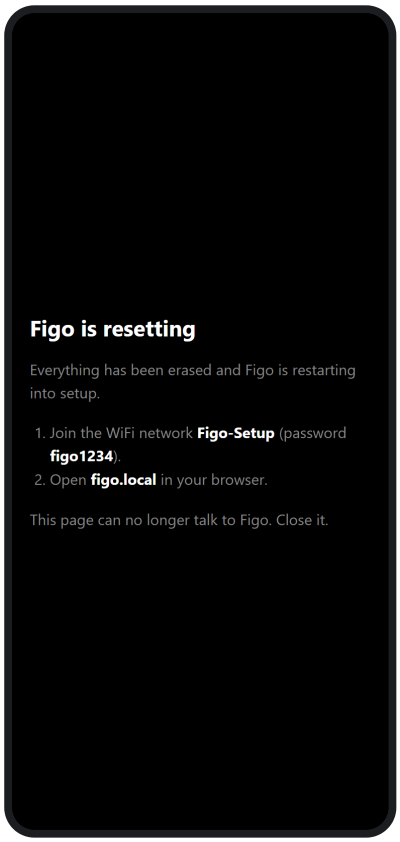
A factory reset erases **everything** Figo has saved — WiFi, screens, settings, face thresholds, provider addresses and any CoinGecko key — and restarts it exactly as it was the first time you switched it on. There is no undo.

From the phone

1. Open **http://figo.local** and go to **SETTINGS**.
2. Tap **Advanced**, and scroll to the bottom: **Start over**.
3. Tap **Factory reset**. The button turns red and asks again: **Tap again to erase everything**. You have five seconds; after that, or if you tap **Cancel**, nothing happens.
4. Tap it again. The phone page is covered by a notice: Figo is erasing itself and restarting.



Armed: the second tap erases.



After: the page cannot reach Figo any more. Close it.

5. Figo restarts with the hello screen, then the setup screen.



Back to the start.

6. Set it up again from chapter 10. It comes back with the four factory screens (chapter 10's table), the star on Bitcoin, and the factory face thresholds.

Things a reset is not needed for

- **Changing WiFi:** Settings → **Forget network** erases only the WiFi and keeps your screens.
- **A stuck or odd-looking screen:** Settings → **Reboot**, or unplug Figo for a few seconds.
- **Installing new firmware:** an update (over WiFi or USB) keeps your screens and settings. Uploading new firmware does *not* reset Figo — even the factory screen list only applies to a Figo that has nothing saved.

If you cannot reach the phone page

You do not need a reset to get back in. If Figo starts up and cannot reach its saved network — you moved house, changed router, or the network is simply gone — it tries for about 18 seconds and then opens its **Figo-Setup** network by itself, and the screen shows the setup details again.



Figo could not reach its saved network, so it asks to be set up.

Join Figo-Setup from your phone and pick the new network, exactly as in chapter 10. Your screens and settings are kept.

If the network disappears while Figo is already running, Figo shows the **WiFi Disconnected** screen instead and keeps retrying. Tap that screen to open Figo-Setup straight away, or unplug Figo and plug it back in (chapter 14).

CHAPTER 16

Appendix: the web API and the settings format

Everything the phone page does, it does through a small web API on Figo itself. You can use it too — from `curl`, a script or a home-automation system — at `http://figo.local`. There is no password: anything on your WiFi can reach it, the same as the phone page.

Routes

This table is generated from the firmware's route list.

METHOD	PATH	WHAT IT DOES
GET	`/api/status`	Mode (STA / AP / NO_WIFI), network name, IP address, firmware version.
GET	`/api/wifi/scan`	Nearby WiFi networks: ssid, rssi, secure. Takes 2 to 4 seconds.
POST	`/api/wifi/forget`	Erase the saved WiFi and restart into setup.
GET	`/api/config`	The whole saved configuration (see the schema below).
GET	`/api/screens/state`	Live values per screen, the current mood band and whether Figo is asleep.
GET	`/api/search/result`	Poll for the answer to the last search or check.
POST	`/api/reboot`	Restart Figo.
POST	`/api/factory-reset`	Erase everything and restart into setup.
POST	`/api/ota/begin`	Start a firmware update. Returns a session token.
GET	`/api/ota/status`	Progress of the running update.
POST	`/api/ota/upload`	The firmware file itself (application/octet-stream).
GET	`/`	The phone app.
GET	`/index.html`	The phone app.
GET	`/update`	Stand-alone firmware update page, for when the app's own Update card is broken.
GET	`/style.css`	Phone app stylesheet.
GET	`/app.js`	Phone app script.
POST	`/api/wifi/connect`	Body {ssid, password}. Join a network and save it.
POST	`/api/config`	Body is ONE section: {system}, {expression_source}, {providers} or {screens}.

METHOD	PATH	WHAT IT DOES
POST	`/api/search/check`	Body {type, symbol}. Ask the provider for one live price.
POST	`/api/search`	Body {q}. Search instruments by name.

Answers are JSON. A refused request answers **400** with `{"error": "…"}`, the message saying exactly what was wrong (for example `screen name must be 1-12 characters for this layout`). While Figo is restarting or installing an update, requests that change something answer **503**.

Searches and price checks take a few seconds, so they work in two steps: `POST /api/search` (or `/api/search/check`) returns `{"ok": true, "seq": N}` straight away, then poll `GET /api/search/result` until its `seq` matches and its `state` is `done` or `failed`.

The settings format

`GET /api/config` returns five sections. `POST /api/config` takes **one section per request**, as a JSON object with that section's name as its only key.

system

```
{ "system": { "brightness": 80, "rotation": 0,
              "sleep_start": "23:00", "sleep_end": "07:00",
              "utc_offset_min": 120 } }
```

FIELD	VALUES
`brightness`	10–100 (percent). Values below 10 are raised to 10.
`rotation`	`0` (cable on the right) or `180`. Applied at the next restart.
`sleep_start`, `sleep_end`	`"HH:MM"`, or both `""` for no sleep. The window may cross midnight.
`utc_offset_min`	Your offset from UTC in minutes, −720 to 840. The phone page sends its own.

expression_source (the star and the face)

```
{ "expression_source": { "screen_id": 2, "bull_threshold": 5,
                         "happy_threshold": 1, "bear_threshold": -2.5 } }
```

FIELD	VALUES
`screen_id`	The `id` of the starred screen, or `0` for none.
`bull_threshold`, `happy_threshold`, `bear_threshold`	Percent. Must satisfy bull > happy > bear.

screens

An array of 1 to 8 screens, in carousel order. Sending it replaces the whole list.

```
{ "screens": [
  { "id": 1, "type": "STOCK", "symbol": "^GSPC", "name": "S&P 500",
    "interval_sec": 900, "layout": "SINGLE", "symbol2": "", "name2": "" },
  { "id": 2, "type": "CRYPTO", "symbol": "bitcoin", "name": "BTC",
    "interval_sec": 120, "layout": "DUAL", "symbol2": "ethereum", "name2": "ETH" }
] }
```

FIELD	VALUES
`id`	1–8. Keep a screen's id when you edit it: Figo keeps its last price for an id whose instrument has not changed.
`type`	`STOCK`, `COMMODITY` (both fetched from Yahoo Finance), `CRYPTO` (CoinGecko) or `FEAR_GREED`.
`symbol`	1–15 characters. A Yahoo symbol (`^GSPC`, `GC=F`, `TEVA.TA`), a CoinGecko coin id (`bitcoin`), or for Fear & Greed `stocks` or `crypto`.
`name`	The title on the panel: 1–12 characters, or 1–10 on a dual screen.
`interval_sec`	Seconds between fetches. At least 60; at least 120 for a dual screen.
`layout`	`SINGLE` or `DUAL` for prices; `SLIDER` or `CIRCULAR` for Fear & Greed.
`symbol2`, `name2`	The right-hand instrument of a dual screen (same rules; name up to 10). Empty otherwise.

providers

```
{ "providers": { "yahoo_host": "query1.finance.yahoo.com",
                 "coingecko_host": "api.coingecko.com",
                 "fng_crypto_host": "api.alternative.me",
                 "fng_stocks_host": "production.dataviz.cnn.io" } }
```

Bare hostnames only. Add `"coingecko_key": "…"` to save a CoinGecko API key, or `"coingecko_key": ""` to remove it; leave the field out to keep whatever is saved. `GET` never returns the key, only `coingecko_key_configured: true/false`.

wifi

Read-only through `/api/config` (`ssid` and `configured`). Change it with `POST /api/wifi/connect` and `POST /api/wifi/forget`.

Live values

`GET /api/screens/state` returns, for each screen, whether it has fetched, whether it is stale, the age of its data in seconds, the price, the change and the currency (plus `price2`, `change_pct2`, `currency2` on a dual screen); the face's current `mood.band` (`BULLISH`, `HAPPY`, `NEUTRAL`, `BEARISH` or `UNKNOWN`); and `sleeping`, which is whether the panel is actually asleep right now.